ICS1511 — Web Application Development Laboratory

Exercise 3: Implementation of Bingo Game using Java Script

Name: Danush K
Reg. No: 3122247001013

Ex. No: 03
Date: 23/07/2026

---

1. Problem Description

1.1 Aim

To design, implement, and verify an interactive, responsive two-player **Bingo Game** web application using **HTML5**, **CSS3**, and **JavaScript** featuring randomized 5×5 grid generation, turn-based input marking across dual boards, and automatic winning condition evaluation.

1.2 Problem Statement & Game Mechanics

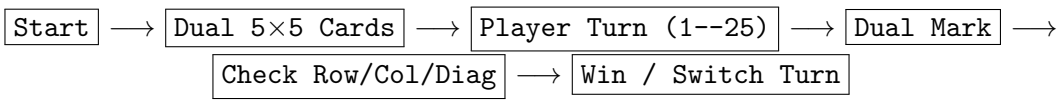
Bingo is a classic probability game played with randomly numbered cards. In this digital implementation, two players compete simultaneously on their respective 5×5 grids:

1. **Game Setup:** Initializes dual Bingo cards populated with random unique numbers (1 to 25).
2. **Start & Reset:** A "Start" button initializes boards; a "Reset" button clears states for a fresh game.
3. **Turn-Based Dual Marking:** Players enter a number (1–25). Upon clicking "Mark", the number is marked on **both** cards.
4. **Winning Condition:** The game continuously evaluates cards for a completed row, column, or diagonal line to declare Bingo.

1.3 System & Software Requirements

- **Operating System:** macOS / Linux / Windows 10 or later
- **Development Environment:** Antigravity IDE / Visual Studio Code
- **Target Web Browsers:** Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge
- **Core Technologies:** HTML5 (Structure), Vanilla CSS3 (Styling & Grid Layout), and JavaScript (Game Logic)

1.4 Game Workflow Architecture



2. Scanned Copy of Observation Content

The handwritten observation note completed during the laboratory session contains the styling definitions, grid layout specifications, DOM selection methods, and instructor verification sign-off.

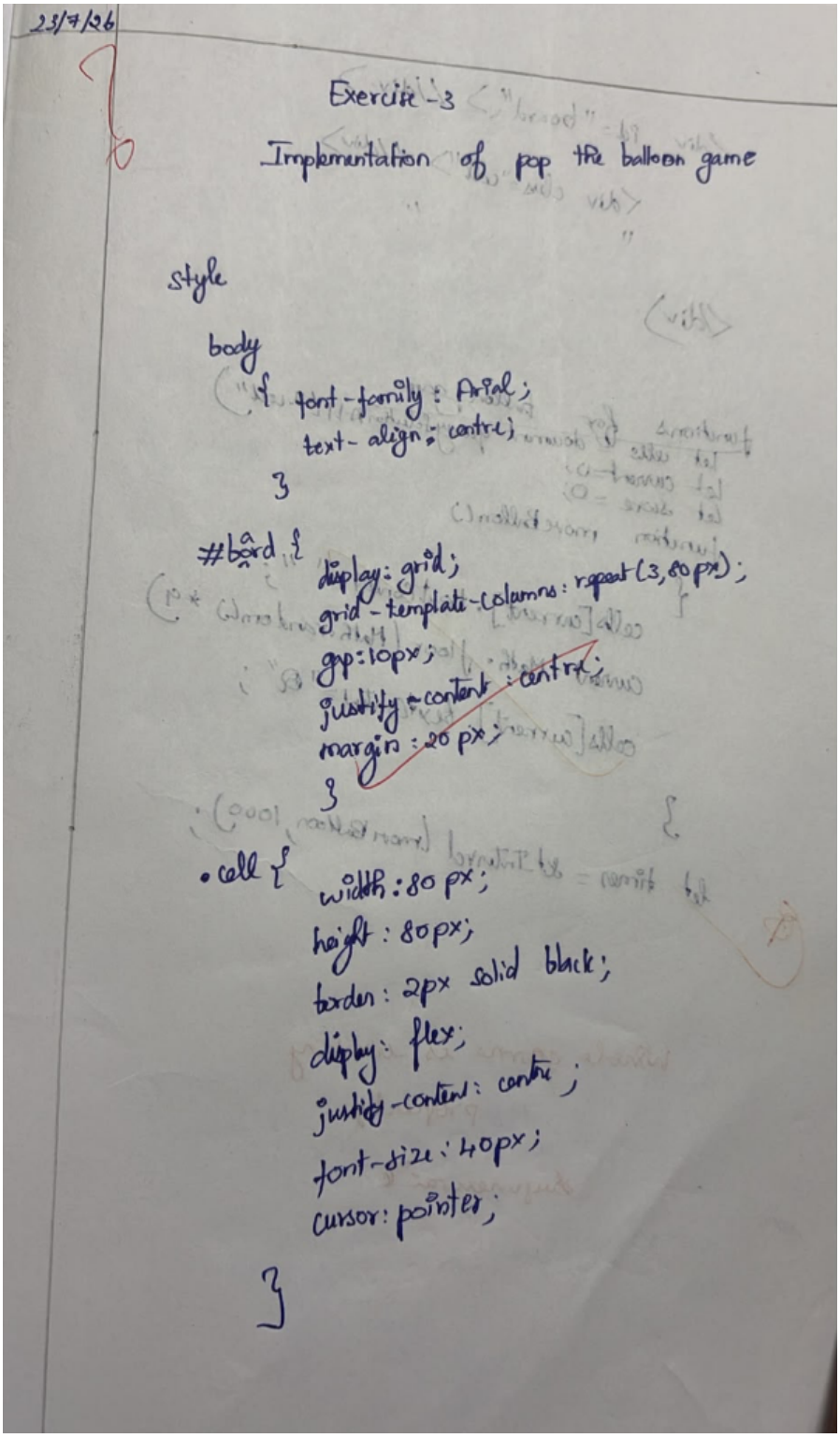


Figure 1: Lab Observation Note — Page 1 (Layout Structure, CSS Grid, and Cell Styling)

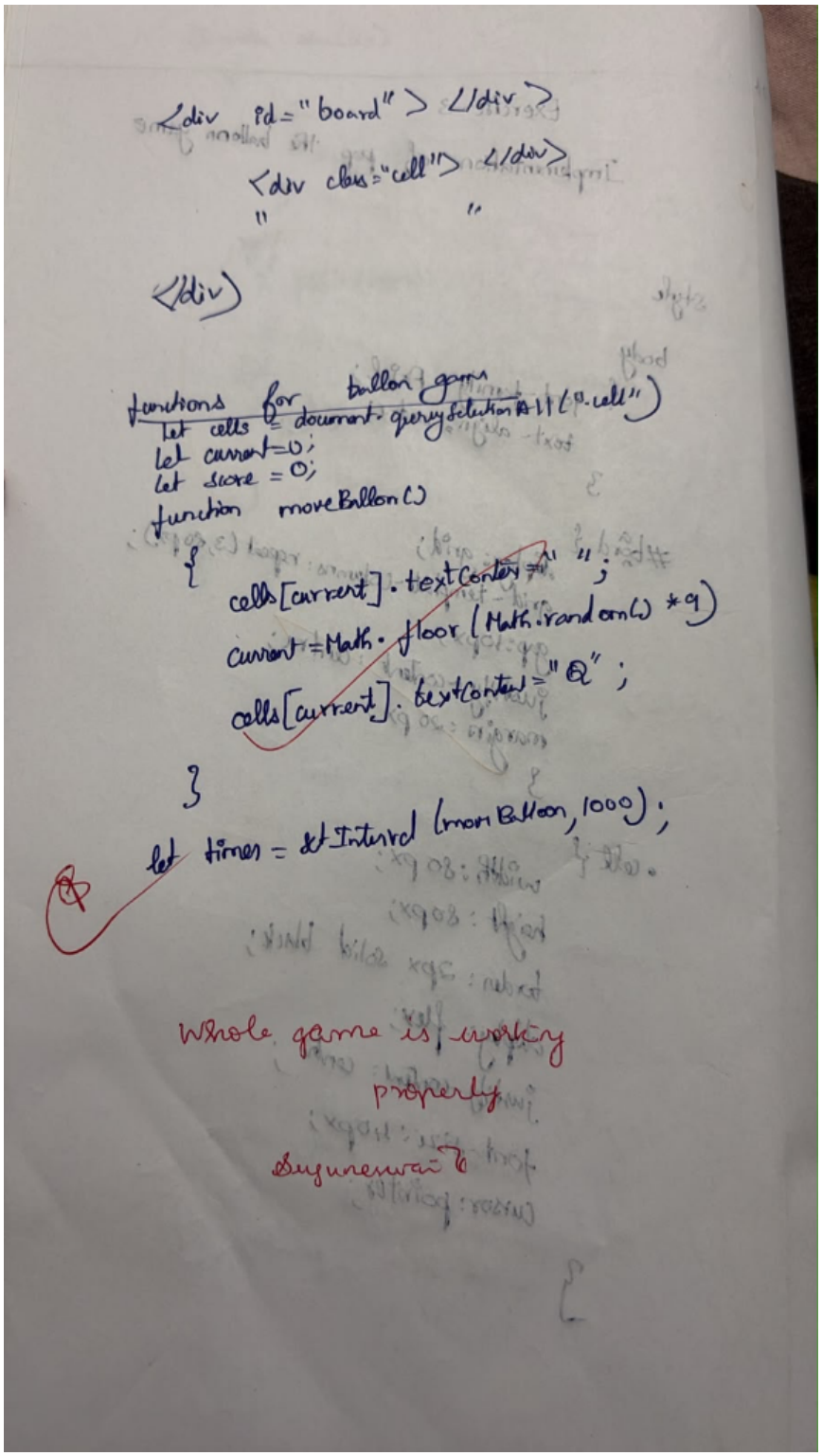


Figure 2: Lab Observation Note — Page 2 (DOM Event Handlers and Verification Sign-off)

3. Source Code

3.1 1. HTML Structure (`index.html`)

Listing 1: Bingo Game Structure (`index.html`)

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bingo Game</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="game-container">
    <h1>Bingo Game</h1>

    <!-- Player Bingo Cards -->
    <div class="boards-container">
      <div class="player-section">
        <h2>Player 1's Card</h2>
        <div id="board1" class="board"></div>
      </div>

      <div class="player-section">
        <h2>Player 2's Card</h2>
        <div id="board2" class="board"></div>
      </div>
    </div>

    <!-- Game Controls -->
    <div class="controls">
      <div class="action-buttons">
        <button id="startBtn" onclick="startGame()">Start</button>
        <button id="resetBtn" onclick="resetGame()">Reset</button>
      </div>

      <div class="input-section">
        <label for="numberInput">Enter a number (1-25): </label>
        <input type="number" id="numberInput" min="1" max="25" disabled>
        <button id="markBtn" onclick="markNumber()" disabled>Mark</button>
      </div>

      <p id="turnStatus">Click "Start" to play!</p>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

3.2 2. Global Stylesheet (`style.css`)

Listing 2: Game Stylesheet (`style.css`)

```css
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
```

```
  font-family: Arial, sans-serif;
  background-color: #d8edd9;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  color: #333;
}

.game-container {
  background-color: #edf7ee;
  border-radius: 12px;
  padding: 25px 35px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.08);
  text-align: center;
  max-width: 650px;
  width: 100%;
}

h1 {
  color: #207548;
  margin-bottom: 20px;
  font-size: 26px;
}

.boards-container {
  display: flex;
  justify-content: center;
  gap: 30px;
  margin-bottom: 25px;
  flex-wrap: wrap;
}

.player-section h2 {
  color: #2d8a59;
  font-size: 17px;
  margin-bottom: 12px;
}

.board {
  display: grid;
  grid-template-columns: repeat(5, 38px);
  grid-gap: 8px;
  justify-content: center;
  background-color: #ffffff;
  padding: 12px;
  border-radius: 10px;
  box-shadow: inset 0 0 5px rgba(0, 0, 0, 0.05);
}

.cell {
  width: 38px;
  height: 38px;
  border-radius: 50%;
  background-color: #28a745;
  color: #ffffff;
  font-size: 13px;
  font-weight: bold;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  user-select: none;
  transition: transform 0.15s, background-color 0.2s;
}

```

```
.cell:hover:not(.marked) {
  transform: scale(1.08);
  background-color: #218838;
}

.cell.marked {
  background-color: #e55353;
  color: #ffffff;
  font-size: 14px;
}

.controls {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 12px;
}

.action-buttons {
  display: flex;
  gap: 12px;
}

button {
  background-color: #62b59d;
  color: #ffffff;
  border: none;
  border-radius: 6px;
  padding: 6px 18px;
  font-size: 14px;
  font-weight: bold;
  cursor: pointer;
  transition: background-color 0.2s;
}

button:hover:not(:disabled) {
  background-color: #4da38a;
}

button:disabled {
  background-color: #b5d5cb;
  cursor: not-allowed;
}

.input-section {
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 14px;
  color: #276749;
  font-weight: bold;
}

input[type="number"] {
  width: 55px;
  padding: 5px;
  border: 1px solid #99c2a6;
  border-radius: 4px;
  text-align: center;
  font-size: 14px;
}

input:focus {
  outline: 2px solid #62b59d;
}

```

```
#turnStatus {
  font-size: 15px;
  font-weight: bold;
  color: #e55353;
  min-height: 20px;
}
```

3.3 3. JavaScript Logic (`script.js`)

Listing 3: Bingo Game Core Mechanics (`script.js`)

```
// Game State Variables
let board1 = [];
let board2 = [];
let marked1 = [];
let marked2 = [];
let currentPlayer = 1;
let gameActive = false;
let calledNumbers = [];

// Helper function to generate shuffled 5x5 card numbers (1 to 25)
function generateCardNumbers() {
  const numbers = Array.from({ length: 25 }, (_, i) => i + 1);
  for (let i = numbers.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [numbers[i], numbers[j]] = [numbers[j], numbers[i]];
  }

  const card = [];
  for (let r = 0; r < 5; r++) {
    card.push(numbers.slice(r * 5, (r + 1) * 5));
  }
  return card;
}

// Render a player's board into the DOM
function renderBoard(boardId, card, marked) {
  const boardEl = document.getElementById(boardId);
  boardEl.innerHTML = "";

  for (let r = 0; r < 5; r++) {
    for (let c = 0; c < 5; c++) {
      const cell = document.createElement("div");
      cell.classList.add("cell");
      const num = card[r][c];

      if (marked[r][c]) {
        cell.classList.add("marked");
        cell.textContent = "X";
      } else {
        cell.textContent = num;
        cell.onclick = () => {
          if (gameActive) {
            document.getElementById("numberInput").value = num;
            markNumber();
          }
        };
      }
      boardEl.appendChild(cell);
    }
  }
}

// Start / Initialize the Bingo Game
function startGame() {
  board1 = generateCardNumbers();
  board2 = generateCardNumbers();
  marked1 = Array.from({ length: 5 }, () => Array(5).fill(false));
  marked2 = Array.from({ length: 5 }, () => Array(5).fill(false));

  calledNumbers = [];
  currentPlayer = 1;
  gameActive = true;

```

```
  document.getElementById("startBtn").disabled = true;
  document.getElementById("markBtn").disabled = false;
  document.getElementById("numberInput").disabled = false;
  document.getElementById("numberInput").value = "";
  document.getElementById("numberInput").focus();

  updateStatus("Player 1's Turn");
  renderBoard("board1", board1, marked1);
  renderBoard("board2", board2, marked2);
}

// Reset the Game to initial state
function resetGame() {
  gameActive = false;
  calledNumbers = [];
  document.getElementById("board1").innerHTML = "";
  document.getElementById("board2").innerHTML = "";
  document.getElementById("startBtn").disabled = false;
  document.getElementById("markBtn").disabled = true;
  document.getElementById("numberInput").disabled = true;
  document.getElementById("numberInput").value = "";
  updateStatus("Click \"Start\" to play!");
}

// Mark the entered number on both players' cards
function markNumber() {
  if (!gameActive) return;

  const inputEl = document.getElementById("numberInput");
  const num = parseInt(inputEl.value, 10);

  // Validate number
  if (isNaN(num) || num < 1 || num > 25) {
    alert("Please enter a valid number between 1 and 25.");
    inputEl.focus();
    return;
  }

  if (calledNumbers.includes(num)) {
    alert(`Number ${num} is already marked! Choose another number.`);
    inputEl.value = "";
    inputEl.focus();
    return;
  }

  // Record number as called
  calledNumbers.push(num);

  // Mark number on both cards if found
  markCard(num, board1, marked1);
  markCard(num, board2, marked2);

  // Re-render boards
  renderBoard("board1", board1, marked1);
  renderBoard("board2", board2, marked2);

  // Check for winning conditions
  const p1Wins = checkBingo(marked1);
  const p2Wins = checkBingo(marked2);

  if (p1Wins && p2Wins) {
    updateStatus("It's a Tie! Both players achieved Bingo!");
    endGame();
  } else if (p1Wins) {
    updateStatus("Bingo! Player 1 Wins the Game!");
    endGame();
```

```
  } else if (p2Wins) {
    updateStatus("Bingo! Player 2 Wins the Game!");
    endGame();
  } else {
    // Switch turn between players
    currentPlayer = currentPlayer === 1 ? 2 : 1;
    updateStatus(`Player ${currentPlayer}'s Turn`);
    inputEl.value = "";
    inputEl.focus();
  }
}

// Mark matching number in a card matrix
function markCard(num, card, marked) {
  for (let r = 0; r < 5; r++) {
    for (let c = 0; c < 5; c++) {
      if (card[r][c] === num) {
        marked[r][c] = true;
      }
    }
  }
}

// Check if any row, column, or diagonal is completely marked
function checkBingo(marked) {
  // Check rows
  for (let r = 0; r < 5; r++) {
    if (marked[r].every(v => v)) return true;
  }

  // Check columns
  for (let c = 0; c < 5; c++) {
    if (marked.every(row => row[c])) return true;
  }

  // Check diagonals
  if ([0, 1, 2, 3, 4].every(i => marked[i][i])) return true;
  if ([0, 1, 2, 3, 4].every(i => marked[i][4 - i])) return true;

  return false;
}

// End the game
function endGame() {
  gameActive = false;
  document.getElementById("markBtn").disabled = true;
  document.getElementById("numberInput").disabled = true;
}

// Update turn / result status message
function updateStatus(msg) {
  document.getElementById("turnStatus").textContent = msg;
}

// Listen for Enter key on the input box
document.getElementById("numberInput").addEventListener("keypress", function (e) {
  if (e.key === "Enter") {
    markNumber();
  }
});
```

4. Output Snapshots

4.1 1. Initial Game State (Cards Generated)

Upon clicking the **"Start"** button, randomized 5×5 grids containing unique values from 1 to 25 are generated for both Player 1 and Player 2, and Player 1 is assigned the active turn.

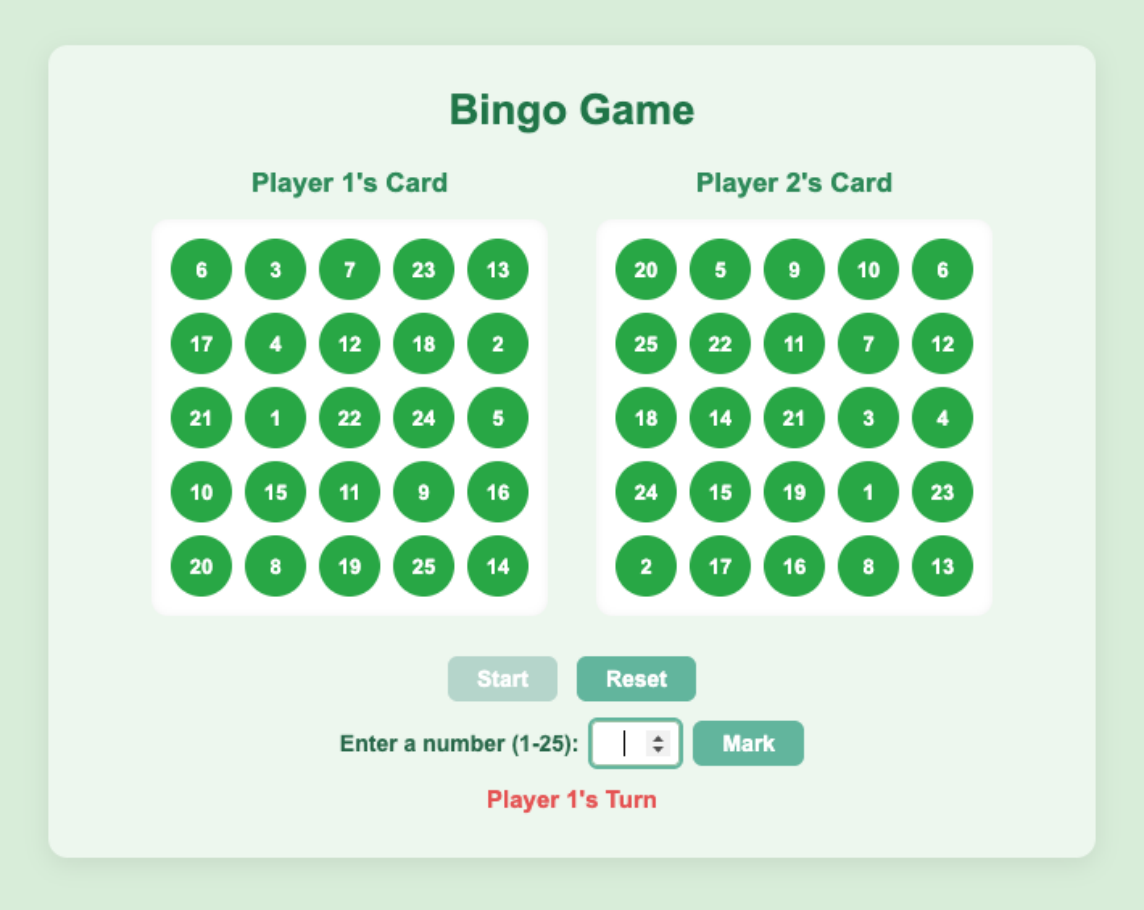


Figure 3: Output Snapshot: Initial State with Randomized Dual Cards (`index.html`)

4.2 2. Game in Progress (Turns and Numbers Marked)

As players take turns entering numbers, the specified values are marked simultaneously on both cards with distinct red circular indicators and the "X" symbol.

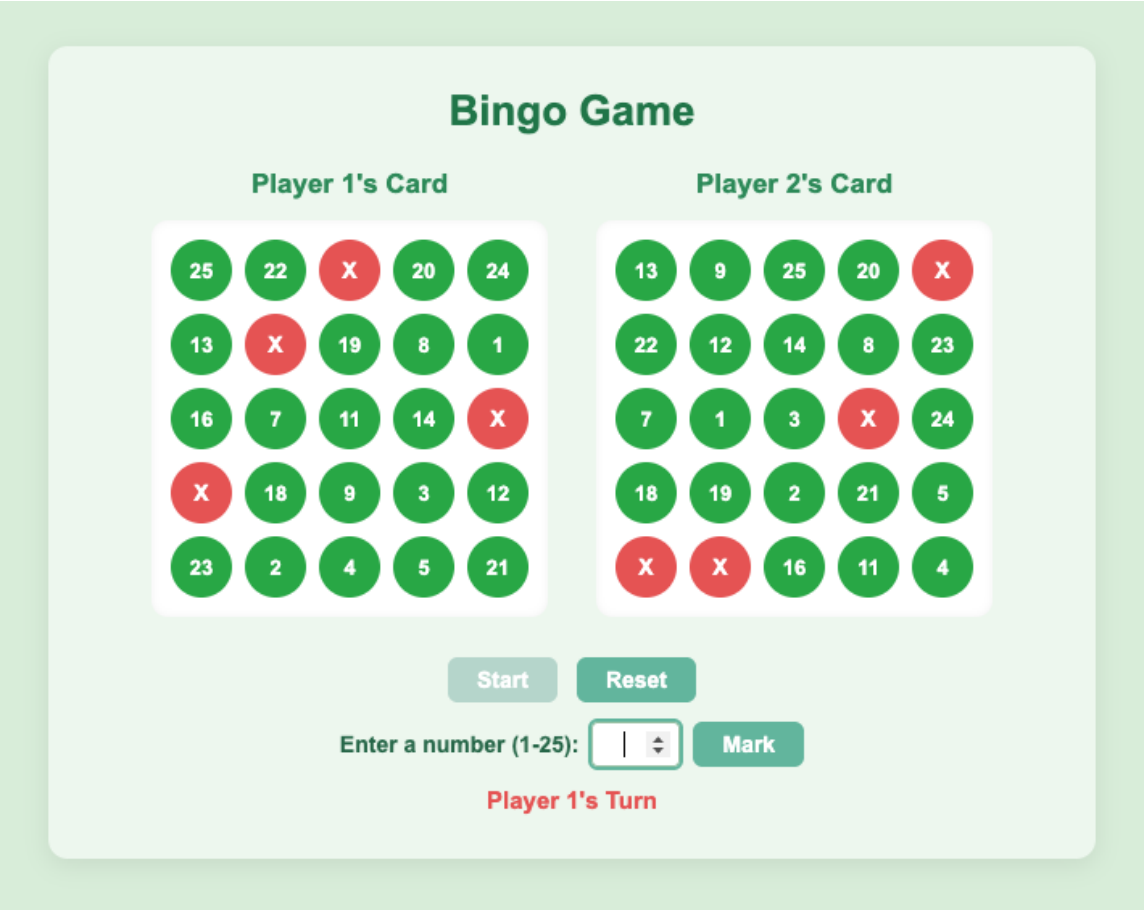


Figure 4: Output Snapshot: Numbers Marked Across Dual Player Cards During Gameplay

4.3 3. Winning State (Bingo Condition Achieved)

When a player completes a full row, column, or diagonal, the winning condition triggers, announcing the winner and disabling the input controls.



Figure 5: Output Snapshot: Winning State Announcement (Bingo Completed)

5. Learning Outcomes

Upon successful design, implementation, and verification of this laboratory exercise, the following core learning outcomes were attained:

1. **DOM Manipulation and Dynamic Rendering:** Learned how to dynamically construct and update HTML elements using JavaScript (`document.createElement`, `innerHTML`, and `appendChild`) to render 5×5 grids from array data structures.

2. **Array Shuffling and Matrix Representation:** Implemented random array permutation algorithms (Fisher-Yates) to generate unique sets of numbers between 1 and 25, structured into 2D matrices for coordinate-based lookups.

3. **Event Handling and Input Validation:** Mastered user event capturing using click handlers and keyboard enter listeners, combined with thorough input validation to reject duplicate entries and numbers outside the allowable range.

4. **Algorithmic Pattern Matching for Win Detection:** Formulated matrix traversal algorithms in JavaScript to evaluate winning conditions across all horizontal rows, vertical columns, and primary/secondary diagonals.

5. **Turn-Based State Management and Responsive CSS:** Acquired proficiency in synchronizing game states across dual players and styling interactive components using CSS Grid and Flexbox with visual feedback.

Result

Thus, a fully functional, two-player interactive **Bingo Game** web application was successfully designed and developed using HTML5, CSS3, and JavaScript in accordance with the laboratory requirements, and the outputs were verified.